

Beyond the Prompt: Hiring Senior Engineers in the 2026 AI Surge



MARTIN LADECKÝ • makeIt & JCON SLOVENIA 2026 • PORTOROŽ

| About me

Technical leader @Commerzbank

15 years of experience in banking

Non-profit experience

10 years of impactful involvement

Czech Java User Group

Dedicated Community Organiser

ExCTO Loono

Previous leadership in HealthTech

The Interview Baseline

30+ Deep Interviews

Throughout 2025, we ran over 30 intensive technical interviews for senior engineers. We bypass the HR filters quickly to observe raw, unvarnished technical reasoning.

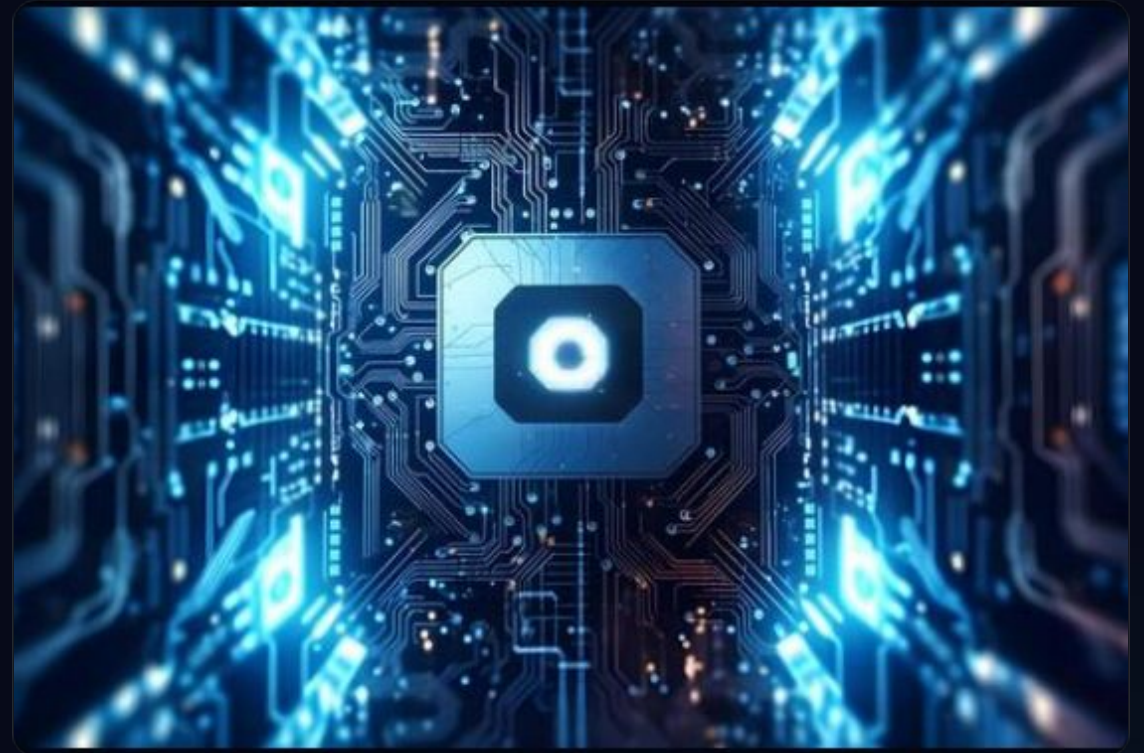
Homework First

Before any live architectural round, candidates must submit hands-on homework. But instead of standard Spring Boot APIs, we took a highly abstract detour to block pattern-matching.

The Un-Cheatable Filter

We abandoned standard enterprise frameworks. Our task focuses on **building complex digital logic components step-by-step** from the ground up, starting with basic transistors and NAND gates.

By stripping away high-level abstractions, we sought to isolate a candidate's mathematical reasoning and state-machine logic. For years, this was our ultimate gatekeeper.



The Hardware Challenge

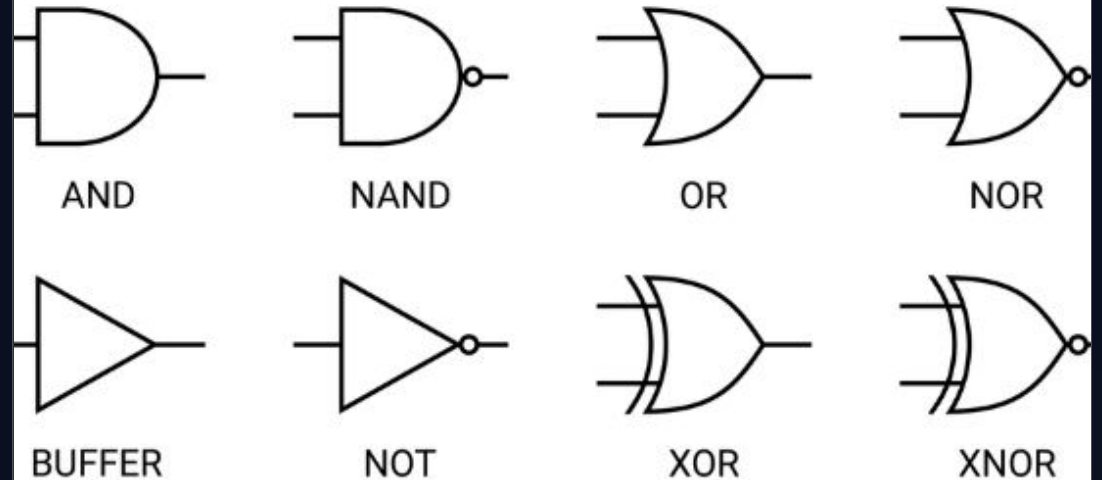
Step 1: Build NAND gates from scratch in Java.

Step 2: Assemble higher logic (AND, OR, XOR) using only NAND primitives.

Step 3: Construct a Gated D Latch & Flip-Flop.

Final Goal: A cascading 4-bit binary counter tracking state cycle-by-cycle.

LOGIC GATE SYMBOLS



| The 2026 Shift

100%

CODE COVERAGE

The Death of Complexity

Suddenly, things changed. Complex logical tasks that candidates previously struggled to finish were being delivered overnight with pristine, 100% code coverage. Multi-modal LLM agents conquered our abstract barriers instantly. The standard take-home had reached a breaking point.

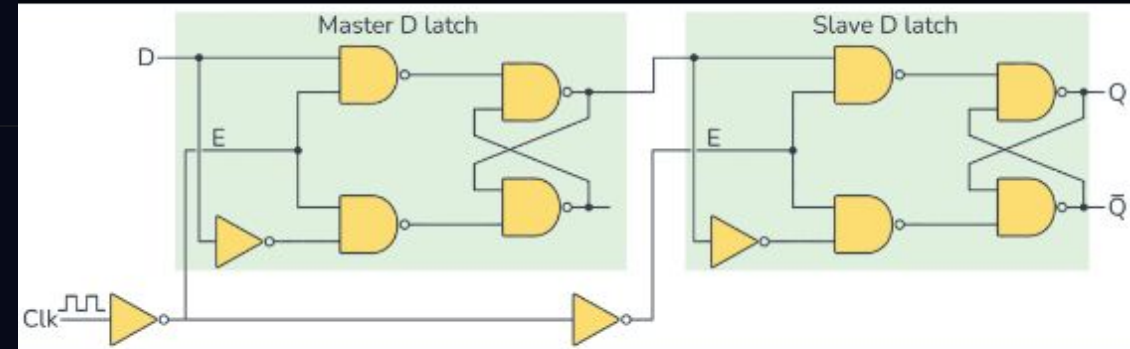
| Exposing the "Synthesizers"

- ⚠️ **The Live Stutter Factor:** In deep walk-through sessions, candidates who submitted flawless logic began to stutter and falter when explaining basic execution traces.
- ❓ **Detached Ownership:** Candidates were completely unsure why their designs worked. They could defend the syntax, but had no conceptual mapping of the underlying states.
- 📄 **The Illusory Prompt:** We were no longer testing engineering excellence; we were measuring a candidate's fluency in copy-pasting code into chatbot.

Case Study: The Hallucination

We tested our abstract task using modern LLMs. It spit out a highly aesthetic solution for a **Master-Slave D-type Flip-Flop**.

But it contained a catastrophic error: it triggered on the "lowering clock edge" instead of the "rising edge".



```
for (int i = 1; i <= 15; i++) {  
    // Simulate Clock Pulse  
    counter.setClock(true); // Rising edge  
    counter.setClock(false); // Falling edge (Flip-Flops trigger here)  
  
    System.out.println(new CounterState(i, counter.getValue()));  
}
```


| I did the homework

DEMO

The Shift to Oral Defense

Why we stopped evaluating the static artifact and
started evaluating live logical verification.

| Our Active Defense

-  **The Live Modification Test:** We accept that they used AI. We start the interview there. "Your code works for 4-bits. Now, scale it to 8-bits live. How does the ripple carry adapt?"
-  **Sifting Candidates Instantly:** True seniors can manipulate the model on the fly. Synthesizers freeze because their custom AI generator isn't connected to the live whiteboard.
-  **Understanding Over Execution:** Code is free; understanding is premium. We shift focus from "Can you write it?" to "Do you fully understand what was written?"
-  **The audit test:** We give them an AI solution with a hidden flaw. Can they catch the architectural hallucination?

The Live Scaling Trap

Hiring Metric	4-Bit Baseline (The AI Task)	8-Bit Scale Up (The Live Test)	The n-Bit Counter (The Bonus)
State Complexity	16 states (Easily simulated)	256 states (Hard to trace manually)	2n dynamic state matrix
AI Capabilities	Pre-trained, high-pattern match	Requires contextual prompts	Requires absolute logic synthesis
Candidate Response	Flawless code, green tests	Sudden friction, stuttering	Reveals actual architectural integrity

A Problem of Choice: Which Language for the Homework?

Our Approach: Seek All Answers



CHATBOTS



The Uniform (Yet Unhelpful) Response



WE ASKED:
**WHICH LANGUAGE
IS THE BEST?**
FOR THIS HOMEWORK.

UNIFORM = NOT HELPFUL?
100% COMPLEXITY STILL EXISTS.

Black Box vs. White Box

Frontend: Black Box

We are comfortable with React components being constructed as a black box. If the component complies, has consistent CSS, matches UX guidelines, and passes standard unit tests, velocity takes priority.

Backend: White Box

The backend requires total comprehension. When production crashes or system constraints buckle under scale, you cannot tell a financial regulator or customer: *"The chatbot said we should write it this way."*

Evolution of Seniority

Syntax & Speed

Pre-2024: Measured by raw typing speed, boilerplate generation, and framework familiarity.

Implementation

2024–2025: Utilizing prompt engineering to string services together and write tests.

Active Auditing

Early 2026: The ability to review, audit, and trace massive AI-generated architectures on the fly.

Architectural Integrity

Late 2026+: First-principles logical thinking, spatial state machines, and complete code accountability.

DEFINING THE 2026 SENIOR

Auditor over Author

Primary skill shifted from code generation to rapid auditing of AI suggestions.

Skepticism First

Treats 100% test coverage as a risk until architectural hallucinations are ruled out.

Mechanical Sympathy

Understanding the "low-level" physics of the system (bitwise, logic, hardware).

Mentally Demanding

Focus on extension and maintenance of systems they didn't fully "write."

*"You cannot tell a regulator that
production crashed because a chatbot
suggested a flawed architecture.
Accountability is the only non-delegable
task."*

Martin Ladecký, Beyond the Prompt

Questions & Discussion



Beyond the Prompt: Hiring Senior Engineers in the 2026 AI Surge

Rate



Download